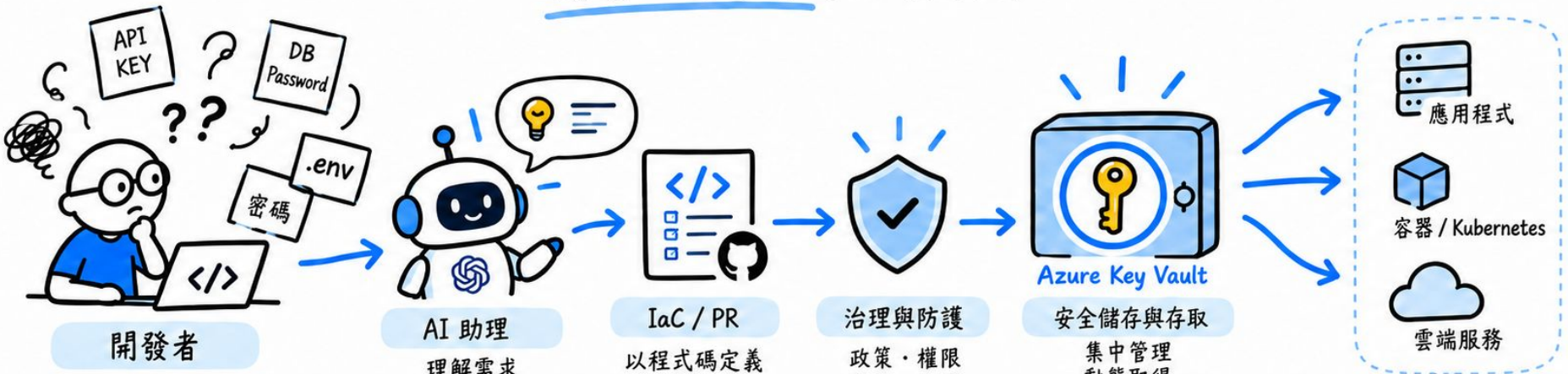
當 Secret 不再是秘密

AI、IaC 與 Azure Keyvault 如何重塑機敏資訊治理

講者：邱繼平（山姆大叔）



Understanding-Driven DevOps 系列 - 1

講者介紹

把治理做成開發者願意使用的產品



邱繼平（山姆大叔）

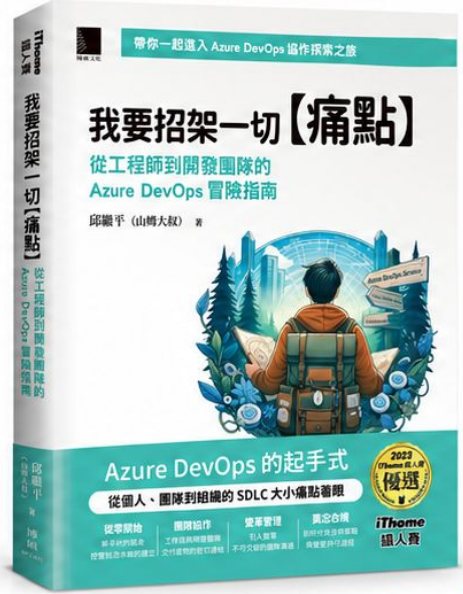
- 13+ 年金融業資訊經驗，近年聚焦 Platform Engineering / DevOps
- Microsoft MVP | Azure DevOps | Kubernetes | Key Vault | Kong
- 主題：自助服務、治理、Golden Path、Developer Experience
- 曾於 Hello World、DevOpsDays Taipei、Platform Engineering Day 分享

我關心的不是把流程變多，而是讓治理真正被使用。

你負責問，我負責答。

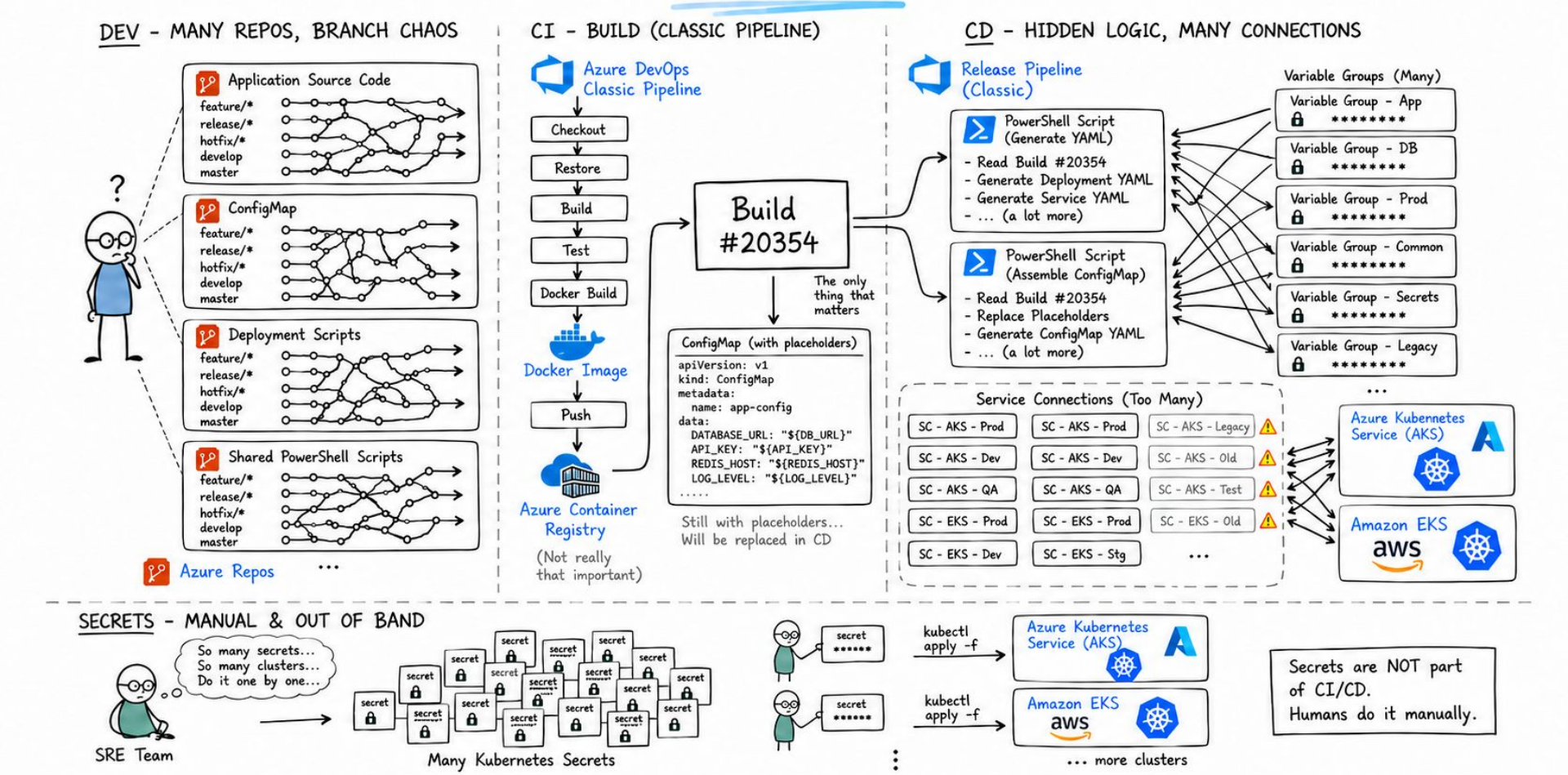
掃 Slido 留下你的問題，最後我會挑幾題回答，也送出幾本書。

Slido 現場提問



最後挑題送書

Understanding-Driven DevOps
改善前的全局



Understanding-Driven DevOps
改善後的全局

① DEV – TRUNK BASED



Azure Repos
my-app-repo

main (trunk)

- ✓ Trunk-based Development
- ✓ Clean & Simple
- ✓ Everything in Git

azure-pipelines.yml

```
trigger: main
stages:
  - build
  - bake
  - publish
  - validate
```

Kustomize Structure

```
base/
  deployment.yaml
  service.yaml
  configmap.yaml
  secret-template.yaml  (no real secret)
overlays/
  dev/   → kustomization.yaml
  sit/   → kustomization.yaml
  uat/   → kustomization.yaml
  prod/  → kustomization.yaml
```

Git Repo contains NO Secrets.

② CI – PRODUCE ARTEFACTS

Azure Pipelines

Checkout → Build → Unit Test → Docker Build → Push Image → Kustomize Bake → Validate → Publish Evidence

Azure Container Registry (ACR)

Docker Image

Kustomize Bake

- Deployment
- Service
- ConfigMap
- Secret Template (no real secret)

Deployment Evidence (Delivery Documents)

DEV	SIT	UAT	PROD
✓ Image: v1.2.3	✓ Image: v1.2.3	✓ Image: v1.2.3	✓ Image: v1.2.3
✓ Manifest Hash	✓ Manifest Hash	✓ Manifest Hash	✓ Manifest Hash
✓ Policy Check	✓ Policy Check	✓ Policy Check	✓ Policy Check
✓ Tests	✓ Tests	✓ Tests	✓ Tests
APPROVED ✓	APPROVED ✓	APPROVED ✓	APPROVED ✓

③ CD – DEPLOY WITH GOVERNED IDENTITY

Azure Pipelines (CD Pipeline)

Receive Deployment Evidence → Validate Evidence (Integrity / Policy) → Get Secrets from Azure Key Vault → Inject Secrets into Manifest → Deploy to Kubernetes → Verify Deployment → Record Evidence

Machine Identity (Governed)

- ✓ Least Privilege
- ✓ Auditable
- ✓ No Human Secrets

Azure Key Vault

- DEV Secrets
- SIT Secrets
- UAT Secrets
- PROD Secrets
- .. ..

↪ Secrets are stored ONLY in Key Vault
↪ Read at deploy time
↪ Only in memory
↪ Never in Git
↪ Never in CI Artifact

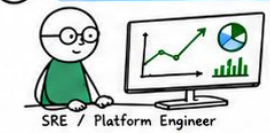
Azure Kubernetes Service (AKS)

Amazon EKS

aws

...

④ PLATFORM TEAM (SRE) – OBSERVE & GOVERN



We don't push secrets.
We don't rebuild YAML.
We govern the platform.
We ensure reliability.
We enable teams.

Delivery Observability Dashboard

Deployments	Change Success	Lead Time	MTTR	Policy Compliance	Evidence Audit
Healthy 98%	95%	2.3 days	45 min	100%	

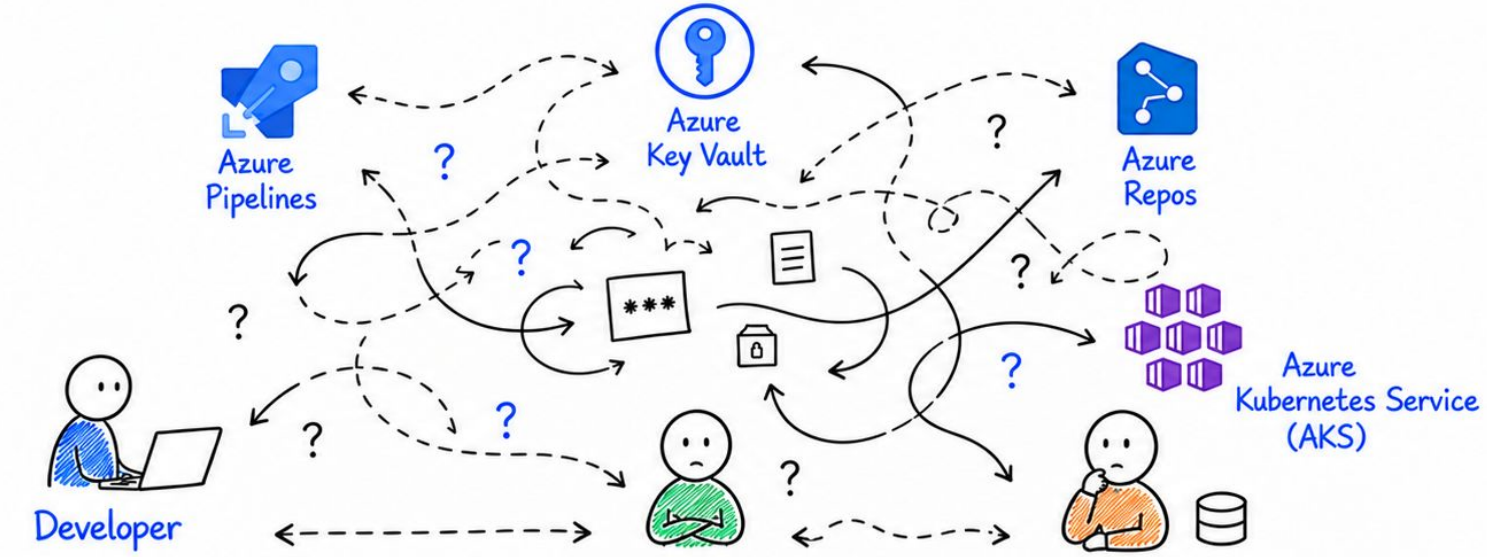
Platform Responsibilities

- ✓ Provide Golden Path
- ✓ Maintain Key Vault & Identity
- ✓ Govern Policies
- ✓ Monitor & Improve
- ✓ Enable Self-Service, Not Manual Work

☆ Deliverable-Driven
Repeatable
Trustable
Productized

Secret 該如何流動與治理呢？

先想想：在你的公司，Secret 是怎麼流動的？



- 需要用到 Secret
- 但不知道去哪裡申請？
- 誰可以給？

- 部署與維運需要 Secret
- 但常臨時被問或找不到來源

- 密碼的擁有者？
- 權限在哪？
- 誰在使用？

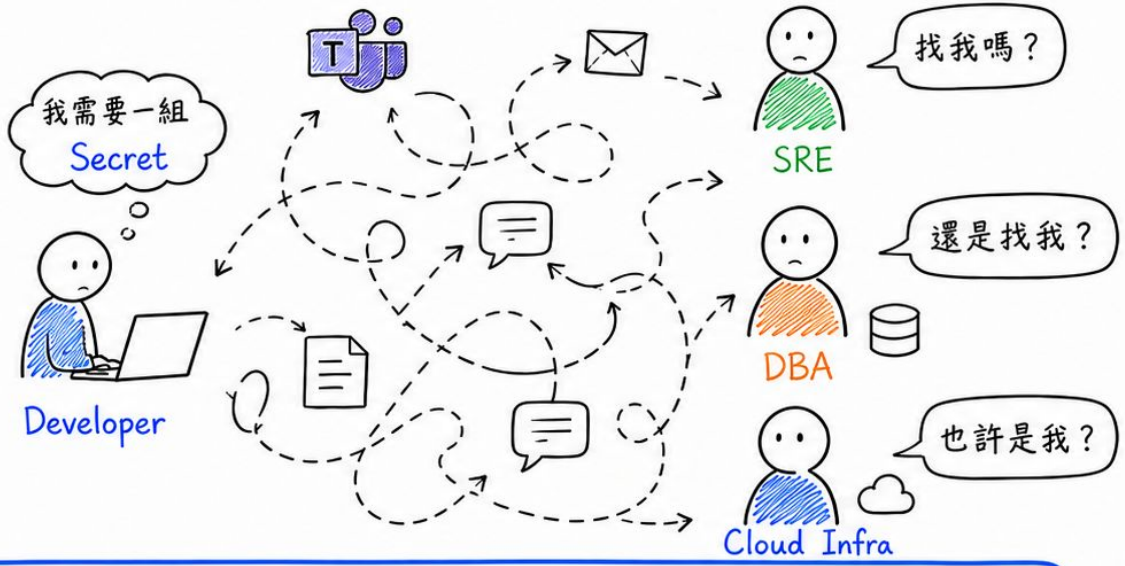
我們以為這是密碼問題

其實，它一直都是協作問題。

我們常聽到的建議

- ✓ 不要把密碼放 Git
- ✓ 不要把密碼寫死
- ✓ 不要把密碼寄 Email
- ...

這些都沒有錯，
但不是我們真正卡住的原因。

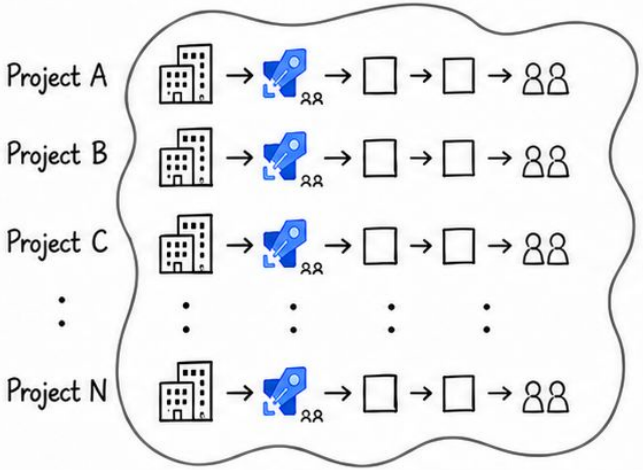


Secret 治理是在設計一條跨部門的工作流。

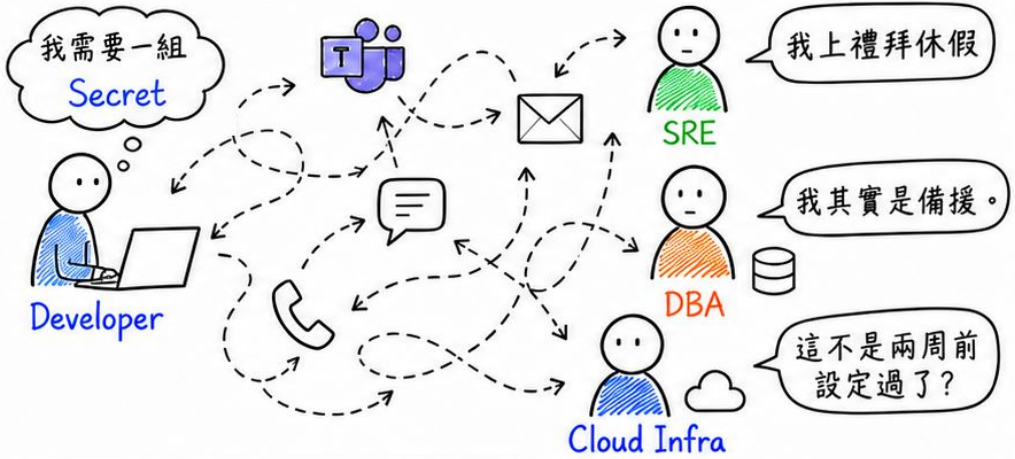
規模化放大了混亂的程度

錯綜複雜的數千次協作

- 100+ Azure DevOps Projects
- 1000+ Pipelines
- 10+ Azure Key Vaults
- 100s Kubernetes Secrets



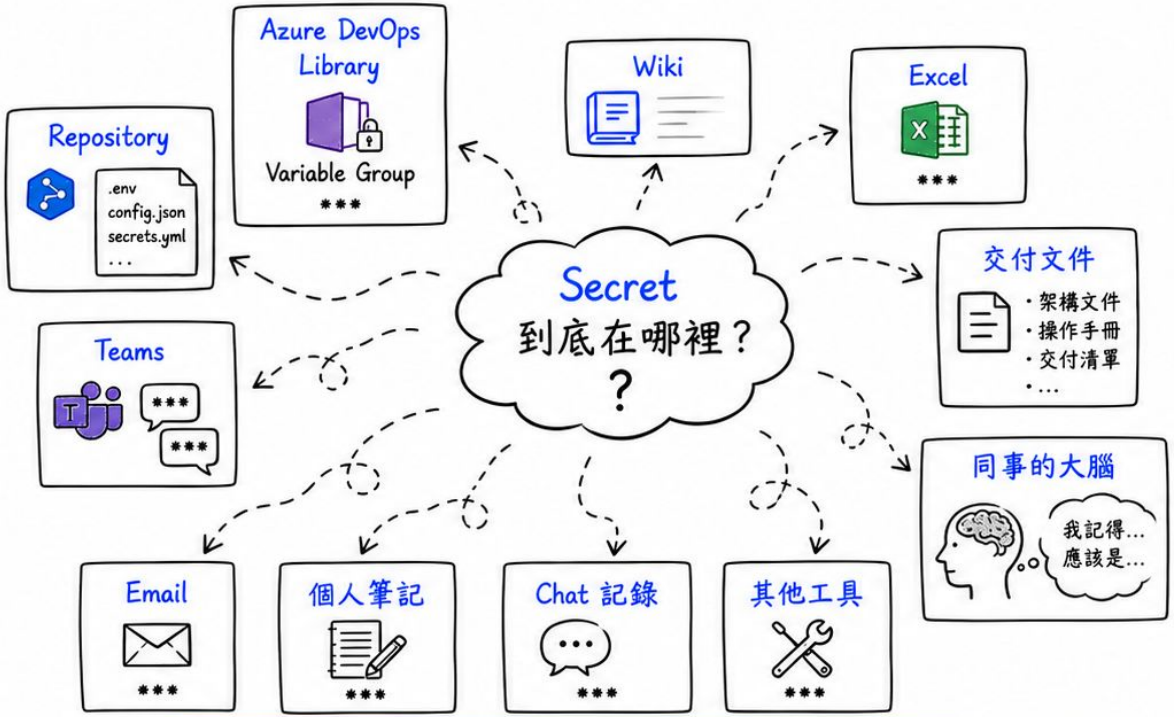
規模化的混亂



當每段協作都有自己的方法，整個組織就沒有答案。

Secret 到底在哪裡？

它可能存在所有你想得到的地方。



每段協作換手的混亂造成了四散的 Secret。

每個人都覺得...



我們 Library 才是真正的來源！



應該是放在 Key Vault，我們說了算！



沒有啦，文件裡寫得很清楚！



問誰？ 我知道啊！

但當你問...



我要建立新的環境，Secret 要去哪裡找？

每個人都有不同答案，
而且都很有自信！

傳說中有些系統有用 Azure Key Vault？

那那堆 secret.yaml 是什麼？





線索一：真的有正確使用的案例

這是一個 Azure Data Factory 專案，
曾由微軟顧問協助導入。

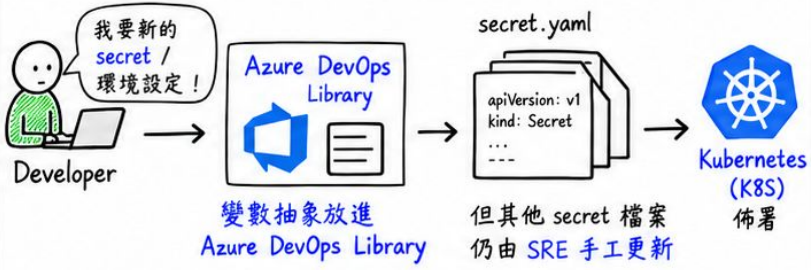


- 從 Key Vault 取得必要的 Secret
- 由 Data Factory 的執行環境使用

這條路徑真的存在，
但不是 K8S 的佈署路徑。

線索二：K8S 專案走的是另一條路

K8S 專案的變數與 Secret 管理，是另一套流程...





真正辛苦的是 SRE，
夾在開發者、DBA 與 Cloud Infra 的摩擦中。

我後來理解：

不是企業沒有 Key Vault，
而是正確使用的經驗，
沒有成為 K8S 專案的標準交付路徑。

研究企業傳說就跟福爾摩斯探案一樣難。

一開始，我要治理的是 Pipeline

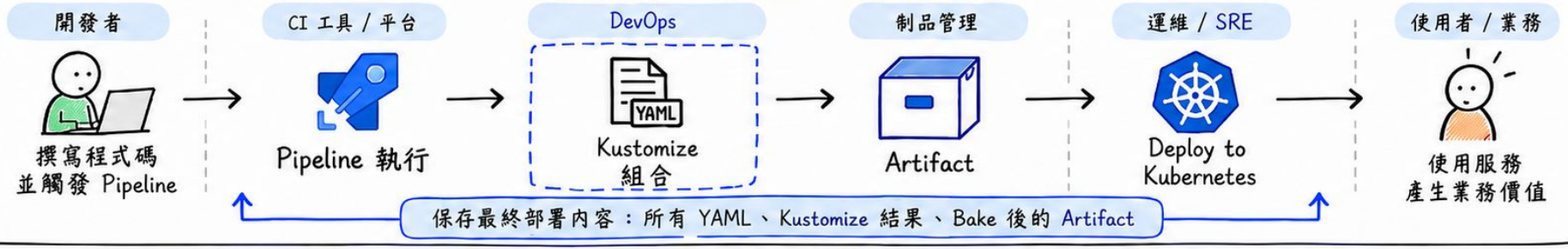
交付流程的重構其實涉及多個角色邊界



組織目標

- 從 Classic Pipeline 轉成 Pipeline as Code
- 每次部署都能留下完整交付證據
- Pipeline 不再是少數人懂的黑盒子

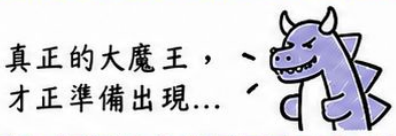
我的想法：可觀測、可追溯的交付



我以為...



我錯了...

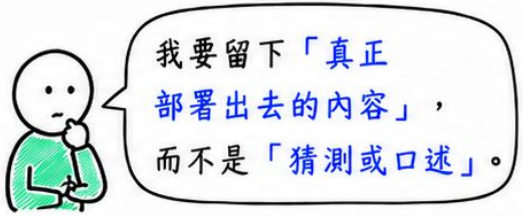




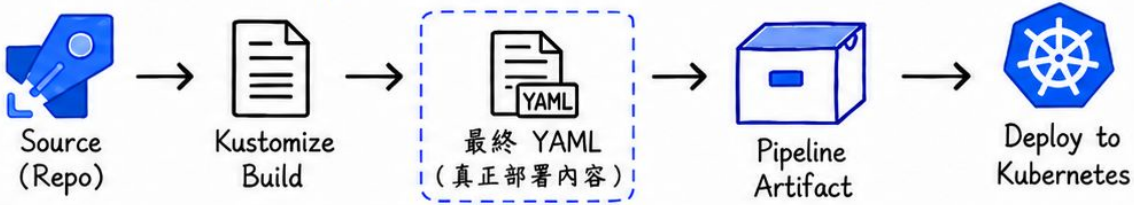
不管如何，拜託不要讓我再管那堆 secret.yaml。

Artifact 應該證明交付內容

如果 Pipeline 是一條交付流程，最後一定要留下證據。

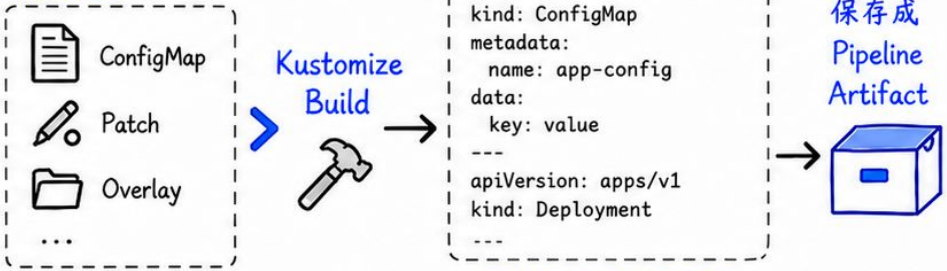


Pipeline 流程（帶著 Kustomize）



如何產生 Artifact

來源內容（人可讀）



（所有內容已展開）

直到... 我開始處理 Secret





Secret 不能進 Repository

事實是不進開發者 Repo，而進了 SRE Repo

工程師的共同原則

Secret 不應該進 Git

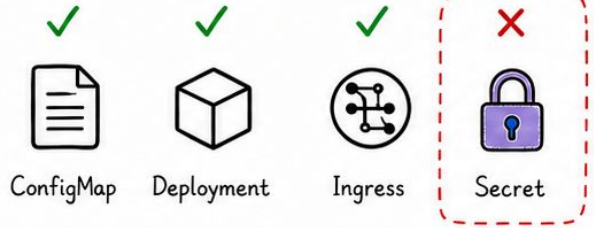




這幾乎是一條
大家都遵守的原則。

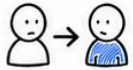
當我們全面導入 IaC...

一切都可以被版本化、審查、追蹤



Secret 成了一個很尷尬的存在。

以前的做法



有人把密碼傳給你



你手動放進去



或交給某個團隊處理

- ✕ 流程沒有被描述
- ✕ 沒有被記錄
- ✕ 沒有被治理

真正的問題不是...

Secret 能不能放 Git



真正的問題是...

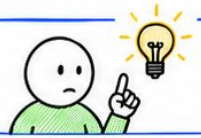




這其實不是技術問題，而是**組織的問題**。

建立一組連線，需要找多少人？

原始流程設計讓各角色協作混亂



有一次，一位開發者只是想建立某個服務的一組連線，需求其實很單純：需要幾組 Secret。

接下來發生的事情...

1 開發者提出需求



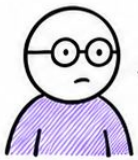
我需要幾組 Secret 建立一組連線。

2 找 Cloud Infra



這個 Secret 應該 DBA 建。

3 找 DBA



我可以提供密碼，但是不是我建立 Secret 。

4 回去找 SRE



好的，我來建立 Secret 。

5 再找 DBA 要值



可以給我密碼值嗎？

密碼值 ******



6 終於完成

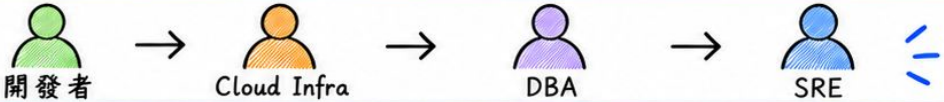


總算可以設定完成了...





花了一個禮拜才完成！





每一個人都認為沒有做錯，因為每個人都完成了自己的工作。

誰應該負責這個密碼？

SRE 是密碼管理者嗎？我 DBA 不認可！！

DBA 的抱怨



這些密碼其實是我們負責的（業務資料庫、帳號、連線字串…）

為什麼最後變成是 SRE 在保管？

雙方都沒有錯！



真正的衝突，不是密碼，而是：**角色沒有被定義！**

SRE 的立場



我們只是負責部署 Kubernetes。

不是我們決定密碼的來源與內容。

以前：大家把所有事情混在一起

- ? 到底誰負責建立？
- ? 誰負責填值？
- ? 誰負責保管？
- ? 誰負責使用？



Secret 在不同團隊之間來回流動，效率低、風險高

也許我們治理的，不應該是 Secret，而是角色。

清楚定義角色與責任，讓 Secret 流動起來

密碼持有者（Owner）	密碼管理者（Manager）	密碼使用者（User）	稽核與治理者（Auditor / Governance）
擁有並產生密碼 決定密碼內容與輪替 （例如：DBA）	管理 Secret 的生命週期 建立、更新、撤銷 （例如：持秘者或專職管理者）	使用 Secret 存取應用或服務 （例如：應用程式、Service Account）	制定規範、監控、稽核 確保合規與風險可控 （例如：InfoSec / GRC）

結論：定義角色，釐清責任，Secret 才能安全、可控、有效率地流動。

問題從來不是放在哪裡

大家最常討論的「存放位置」，其實都太後面了...

真正應該先回答的，是這五個關鍵問題（順序很重要）

1. 誰提出需求？



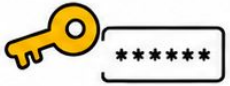
明確定義需求與用途

2. 誰建立 Secret Container？



建立 Secret 的容器（例如：Key Vault、Vault、K8s Secret）

3. 誰填入真正的 Secret Value？



提供並填入真正的敏感值（密碼、憑證等）

4. 誰負責審核？



確認正確性、合規性與存取權限

5. 最後誰會使用？



在 Runtime 或系統中安全地使用 Secret

如果這五個問題沒有答案，

Secret 放在哪裡，其實沒有太大差別。

因此，我們的思維轉變是：

過去：
把 Secret 當成一個物件，儲存在某個系統裡。



現在：
把 Secret 當成一條工作流，由不同角色共同完成。



從這裡開始，我們重新設計了整個治理模型。

結論：Secret Management 的本質，不是「放在哪裡」，而是「誰、在什麼時機、用什麼流程」來處理它。

開發者不需要知道密碼

但仍然可以完成工作

開發者：定義 Secret Name 與使用方式

```
env:
 - name: DB_HOST
   valueFrom:
    secretKeyRef:
     name: app-db-secret
     key: host
 - name: DB_PASSWORD
   valueFrom:
    secretKeyRef:
     name: app-db-secret
     key: password
```

- 定義 Secret 名稱
- 定義需要的 Key
- 描述如何在程式中使用





K8s Manifest (YAML)

將 Secret Name 與 Secret Value 做徹底分開

Secret Name
（公開資訊）



由開發者定義
可以放 Git

Secret Value
（敏感資訊）



由持密者填入
不放 Git

持密者：填入真正的 Secret Value

- 提供/產生真正的密碼
- 填入對應的 Key
- 確保密碼的正確性與安全性





Secret (YAML)

Pipeline：部署時才組合兩者



取得 Manifest（含 Secret Name）

取得 Secret Value（從受控系統）

組合並產生最終 YAML

部署到 Kubernetes



部署過程中，Secret Value 只在受控環境中存在，不被記錄、不被保存。

這樣的設計，讓每個人都更安全，也更專注



開發者：專注在功能與定義，不碰觸密碼



持密者：掌控密碼生命週期與輪替



整個流程：責任清楚、風險降低、效率提升



開發者不需要知道密碼，也可以完成部署。因為每個人只知道自己應該知道的事情。

重新畫出 角色拓樸

不是工具的最佳實踐，而是角色的最佳協作

Developer
需求描述者



- 描述需求
- 定義 Secret Name
- 指定使用方式
- 不接觸任何 Secret Value

負責：描述需求與使用方式

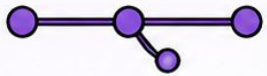
Reviewer
變更理解者



- 閱讀與理解變更
- 檢查合理性
- 確認安全與合規
- 不接觸任何 Secret Value

負責：理解即將發生的變更

Azure DevOps Pull Request
檢查與建立者



- 提供審查流程
- 政策檢查（Policy）
- 自動化檢查（Lint/Scan）
- 建立審計軌跡
- 自動化建立資源

負責：提供治理與建立能力

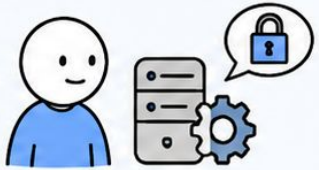
Secret Owner
密碼持有者



- 提供／產生 Secret Value
- 填入對應的 Key
- 控管生命週期與輪替
- 檢視存取者

負責：填入並維護真正的 Secret

Runtime
執行使用者



- 在部署時取得 Secret
- 僅在執行時使用
- 不儲存、不記錄
- 最小化權限存取

負責：取得並安全使用 Secret

端到端工作流



治理與稽核 每個步驟都有清楚的責任人、可追蹤、可稽核、可回溯。

Step 1：宣告需求



整個工作流的第一步，不是建立 Key Vault，而是「宣告需求」。
我們希望把需求變成一份可理解、可審查、可追蹤的描述。

1 宣告需求 → 2 提交 PR → 3 建立容器 → 4 由持密者填值 → 5 部署取得 → 6 部署完成 → 7 安全使用

開發者：描述需求（用 Bicep Param）



開發者需要提供的資訊

- 使用的 Key Vault 名稱或 ID
- 需要的 Secret Name 清單
- 應用程式名稱 / 環境

AI Skill：訪談與整理



AI 訪談範例問題

- 這些 Secret 要用在哪個應用程式？
- 需要幾個 Secret？分別是什麼用途？
- 在哪些環境需要？（UAT / Prod）

產出：需求描述（Bicep Param）

```
param keyVaultName string = 'kv-app-uAT-001'
param secrets array = [
  {
    name: 'Db-Connection-String'
    description: 'Database connection string'
  },
  {
    name: 'Api-Client-Secret'
    description: 'API client secret for third party'
  }
]

param environment string = 'UAT'
param application string = 'OrderService'
```

這份檔案的意義

- 不包含任何 Secret Value
- 可被審查
- 可被追蹤
- 可被稽核
- 可被重現

params.main.bicepparam
（儲存於 Git）

Step 2：讓變更可以被理解



Pull Request 的目的，不只是 Code Review，
而是讓整個變更**被理解**、**被驗證**、**被審核**。

1 宣告需求 → **2 提交 PR** → 3 建立容器 → 4 由持密者填值 → 5 部署取得 → 6 部署完成 → 7 安全使用

Pipeline 在 PR 階段自動執行

1. Validate（驗證語法與架構）

✅ 確認 IaC 能成功部署

```
$ az deployment sub validate
   --template main.bicep
   --parameters params.main.bicepparam

✅ Validate passed
All resources are valid.
```

檢查項目

- Bicep 格式是否正確
- 參數是否完整
- 資源是否存在依賴問題
- 權限與參考是否有效

2. What-If（預覽變更內容）

✅ 如果 Merge，世界會變成什麼樣子？

```
$ az deployment sub what-if
   --template main.bicep
   --parameters params.main.bicepparam

## Changes to be deployed
+ Microsoft:KeyVault/vaults/kv-app-uAT-001
+ Microsoft:Authorization/roleAssignments
~ Microsoft:KeyVault/vaults/kv-app-uAT-001
  properties.accessPolicies[+1]
- Microsoft:KeyVault/vaults/old-kv (no longer exist)
```

變更類型

+ 建立（Create）	~ 更新（Modify）
- 刪除（Delete）	= 保持不變（No Change）

Reviewer 審核的不只是程式，而是基礎設施即將發生的變化



我可以清楚看到：
- 新增了什麼？
- 更新了什麼？

帶來的改變

- ✅ 變更透明化：所有改動一目了然
- ✅ 風險可控化：在 Merge 前發現問題
- ✅ 溝通清楚化：用事實而非猜測討論
- ✅ 責任明確化：每一次變更都有紀錄
- ✅ 規則迭代化：持續演進自動檢查項目

Step 3：先建立容器，再放內容



PR 通過後，Pipeline 會先建立「容器」，
真正的 Secret Value，則交由持密者填入。

1 宣告需求 → 2 提交 PR → **3 建立容器** → 4 由持密者填值 → 5 部署取得 → 6 部署完成 → 7 安全使用

Pipeline 執行：建立 Secret Container（只有 Name，沒有 Value）

依 Bicep 參數建立

```
param keyVaultName string
param secrets array = [
  { name: 'Db-Connection-String' },
  { name: 'Api-Client-Secret' }
]
...
```

只根據 Name 建立，
不包含任何 Value

Azure Key Vault
kv-app-uAT-001

Secret Container（只有 Name）

Db-Connection-String（空的）	Api-Client-Secret（空的）
Name: Db-Connection-String	Name: Api-Client-Secret
Value: —	Value: —

目前狀態：容器已建立，但所有 Secret 都是「空的」

此時並沒有發生的事

- 沒有任何 Secret Value（沒有密碼被產生）
- 平台沒有拿到密碼（SRE 不知道 Value）
- 開發者沒有拿到密碼（Dev 不知道 Value）
- 程式碼庫沒有存密碼（Repo 安全無密碼）
- 暫時無法被應用程式使用（因為沒有 Value）

安全與職責已經分離完成！
容器建立 ≠ 密碼提供

Step 4：由真正的持密者填值



容器建立完成後，由真正的持密者填入 Secret Value。
每個人只接觸自己該接觸的資訊，協作更安全、更高效。

1 宣告需求 → 2 提交 PR → 3 建立容器 → **4 由持密者填值** → 5 部署取得 → 6 部署完成 → 7 安全使用

1 Developer 建立工作項目（Azure Boards）



Azure Boards

標題：申請 DB 連線相關 Secret

描述：
已建立 Key Vault 與 Secret Container，
請 DBA 提供連線密碼。

Key Vault： kv-app-uAT-001

Secret 名稱：
- Db-Connection-String
- Api-Client-Secret

Secret 連結（只有得到名稱，看不到值）

Azure Key Vault 連結
https://kv.azure.net/secrets?vault=kv-app-uAT-001
（需要在組織內身分驗證才能開啟）

Developer 可以看到 Secret Name，
但看不到任何 Value。

2 持密者打開連結，填入 Secret Value



透過連結開啟 Azure Key Vault
https://kv.azure.net/secrets?vault=kv-app-uAT-001
（已驗證身分與權限）

Azure Key Vault
Vault： kv-app-uAT-001

Secret Name	Value	狀態
Db-Connection-String	••••••••••••••••••	已更新
Api-Client-Secret	••••••••••••••••••	已更新

持密者只負責填入 Value，不需要知道其他資訊。

3 填值完成，自動記錄與通知

自動記錄（Audit Trail）

- 誰填入（User / Service Principal）
- 填入時間（Timestamp）
- 填入的 Secret 名稱
- 來源 IP / Client
- 操作（Set Secret Value）



所有交接都有記錄可追蹤、可稽核。

Step 5：系統取得，不是人取得



部署的時候，讓「系統」自己取得 Secret，
而不再需要任何人拿到真正的密碼。

1 宣告需求 → 2 提交 PR → 3 建立容器 → 4 由持密者填值 → **5 部署取得** → 6 部署完成 → 7 安全使用

CD Pipeline：在部署時才取得 Secret

運作流程圖



誰會接觸到 Secret？

Developer	✗ 看不到
Reviewer	✗ 看不到
CD Pipeline（系統）	✓ 部署時才讀取
Runtime（應用程式）	✓ 執行時使用
持密者（DBA / SRE / Infra）	✓ 填入時接觸

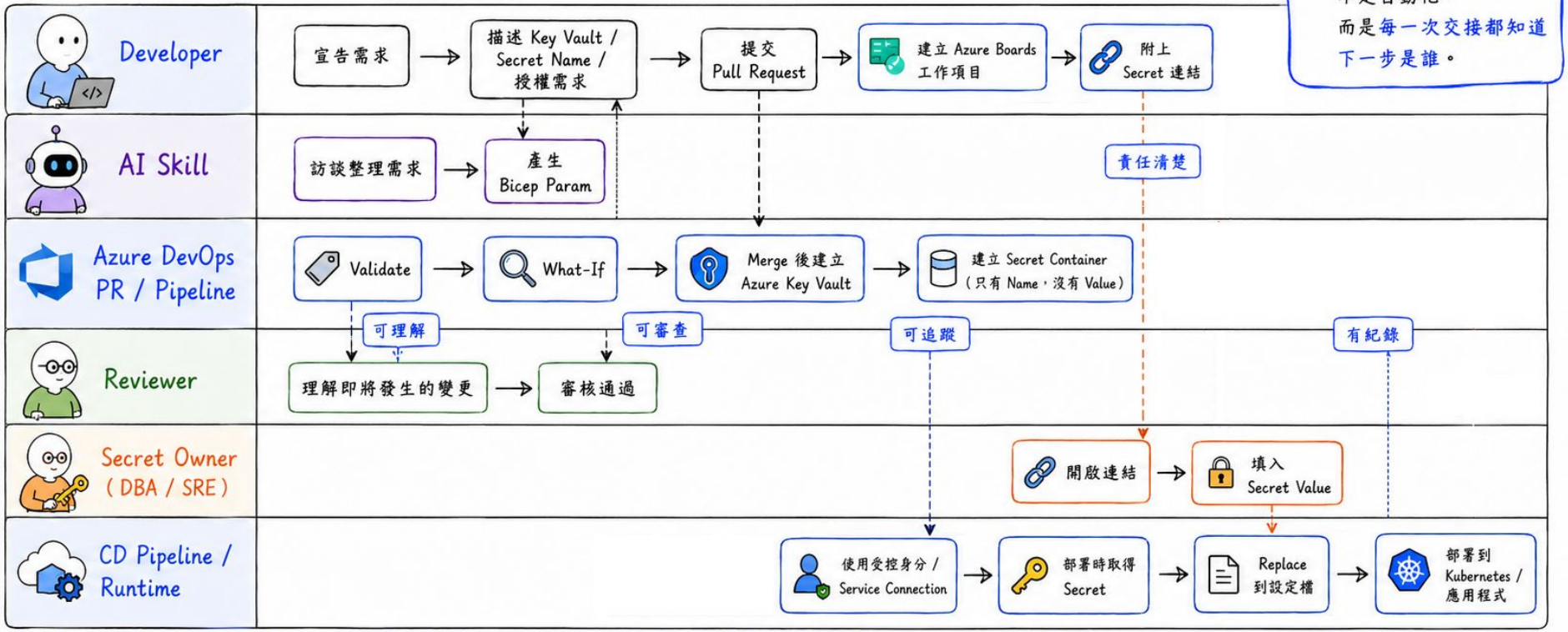
這樣的設計帶來的價值

- ✓ 人逐步退出 Secret 傳遞流程
- ✓ 系統完成原本需要人完成的事
- ✓ 安全性提升，風險大幅降低
- ✓ 權責清楚，流程可追蹤、可稽核

完整的 Secret Workflow

把前面所有內容收斂成一張泳道圖

> ⭐ 這條工作流最大的特色，
> 不是自動化，
> 而是每一次交接都知道下一步是誰。



角色清楚
每個人只負責自己的事情

交接有序
每一步交接都知道下一步是誰

全程可追蹤
從需求到部署，都有記錄可追溯

持續可演進
流程可優化，實踐持續改善

可以被理解、可以被治理
技術與治理並重，安全又高效

從意圖到 Pull Request

AI Skill 協助工作，而不是取代工作

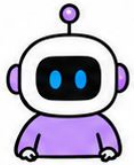
1 提出意圖（自然語言）



Developer 不需要...

- × 學習 Bicep 語法
- × 查命名規則
- × 研究標籤策略
- × 記住目錄結構
- × 記得 PR 流程

2 AI Skill 對話式詢問



AI：我需要再確認幾件事：

① Task 建了嗎？
② Branch 要叫什麼？
③ Stage 是哪個？
④ 國家 / 區域？
⑤ Namespace 是什麼？
⑥ 需要哪些 Secret？
⑦ 需要哪些 Tag？
⑧ 成本中心是？
⑨ 其他備註？

Developer：逐一回答...

3 AI Skill 協助完成準備工作

1. 建立 Branch
   feature/kv-prod-eastasia-db-api
2. 產生 Bicep Param
   （bicepparam）
   - 命名符合規則
   - 結構符合標準
3. Commit
   包含 bicepparam 檔案
   與說明文件
4. Push
   推送到遠端 Repository
5. 建立 Pull Request
   包含描述與變更概要

4 Azure DevOps Service (Repo)

Azure Repos (Git)

```
infra/
  keyvault/
    prod/
      eastasia/
        kv-prod-eastasia/
          kv-prod-eastasia.bicepparam
          README.md
          .gitignore
          ...
```

Pull Request
PR #125

- Title
- Description
- Change Summary (What-If)
- Reviewers
- Linked Azure Boards Task

5 人來審核與部署



審查程式碼
與 What-If



檢視權限與
安全設定



透過 Boards
填入 Secret
（不在 PR 中）



合併後自動執行
建立 Key Vault
與 Secret Container

AI 做了什麼？

- ✓ 理解需求
- ✓ 產生檔案
- ✓ 詢問確認
- ✓ 準備 PR

人與流程仍然負責

- ✓ 檢查與審核
- ✓ 合併與部署
- ✓ 授權與批准
- ✓ 治理與追蹤

AI 的定位： 降低理解成本，提前完成準備工作，加速，而不越權。

AI 降低理解成本

讓每個人更容易理解，做出更好的決策

常見的誤解

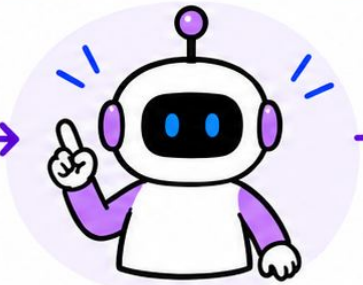
AI 可以取代工程師？
AI 會自己決定？
AI 可以自己部署？

✕ 答案：不是

輸入：需求與情境

- 我要建立 Key Vault
- 所在國家 / Stage
- Namespace
- 需要的 Secret
- Tag / 成本中心

AI 的定位：降低理解成本



輸出：可理解的結果

- 符合規範的參數
- 清楚的 What-If
- 風險與影響說明
- 建議與提醒

真正的價值

把大家腦中的知識，轉成每個人都能理解的工作流。

AI 不是治理模型，而是理解的加速器。

決策與責任，仍然在人



Developer
提出需求
描述意圖



Reviewer
理解變更
審核風險與影響



Security
檢視權限
與安全設定



Secret Owner
填入 Secret Value
管理機密內容



Platform / SRE
維護平台規範
與基礎設施

定義規則的人是人、審核的人是人、授權的人是人、最終負責的人仍然是人

AI 帶來的正向影響

更容易理解
將規範轉成可理解的語言

更快準備
減少重複整理與查找時間

更一致溝通
統一命名、標籤、結構、權限邏輯

更可治理
流程透明、可追蹤、可稽核

更高效團隊
專注在決策與價值創造

自助化建立 Azure Keyvault 解放雲端基礎設施管理員

逐件審查、規則與護欄加高到最終全自助建立

初期：逐件審查

已有基本的自動化檢查，但仍需人工審查。

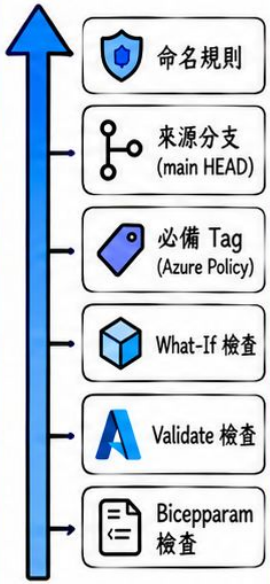


中期：護欄逐步加高

加入更多檢查與規則，降低人工判斷，Reviewer 只需確認全部為綠燈。



規則與護欄加高



最終：全自助建立

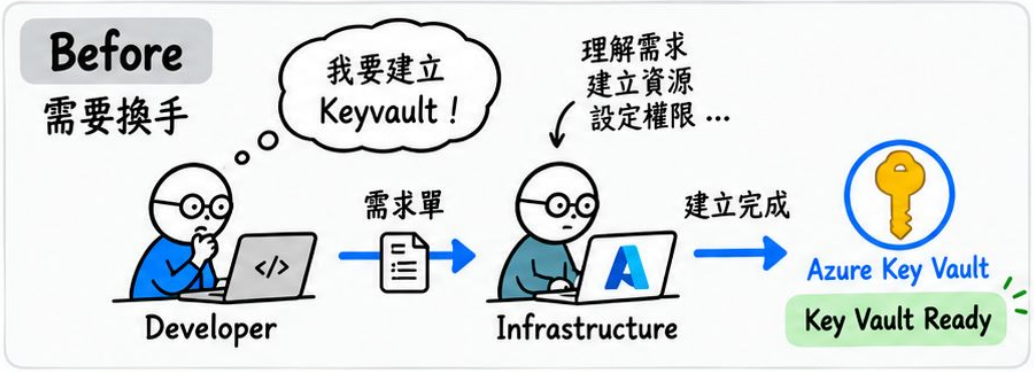
所有檢查通過後，自動建立 Key Vault，不再需要任何人工審查。



2026/09/15 移除 Reviewer 流程，只要符合所有檢查規則即可自行建立

第一條價值流

從需求到「Key Vault Ready」





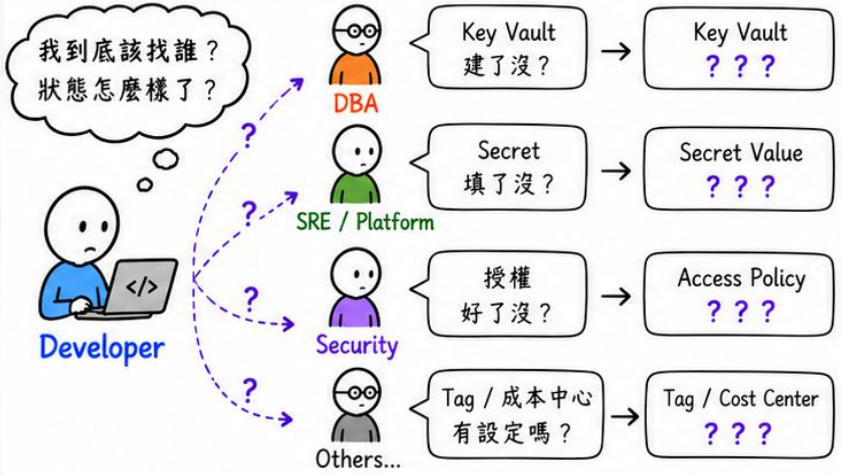
Metric	Before	Self-Service
Lead Time	~1 day	~7 min
Human Touch	~30 min	~5 min
Queue / Wait	~1 day	≈ 0
Handoff	Developer → Infra	0
Self-Service	0%	100%

從過去換手的摩擦，改善為全自助的流程，
可以看見**價值順暢的交付**。

從找人，變成走流程

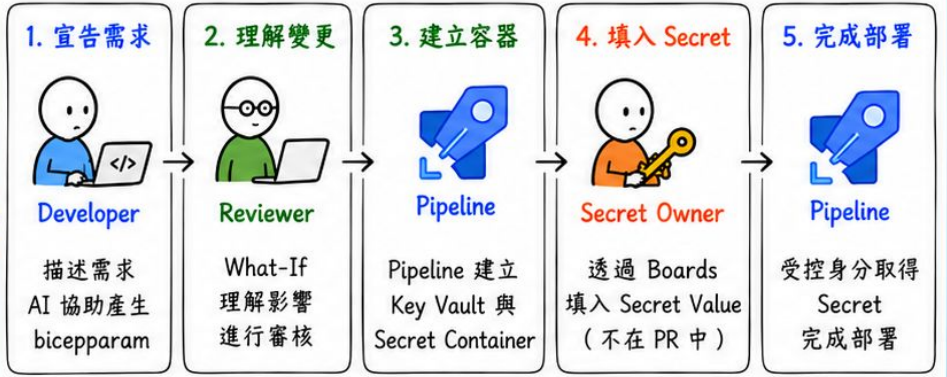
我們不是把 Secret 管得更安全，而是把工作流變得更順

Before：到處找人，充滿不確定



- 每一次建立環境，都像解題
- 大量重複溝通，依賴個人記憶與經驗
- 進度不透明，容易遺漏與延誤
- 缺乏紀錄，難以追蹤與治理

After：走流程，每個人都知道下一步



每一步都有紀錄 | 每一步都有追蹤 | 每一步都有治理 | 整體流程透明可視

- ✓ 清楚的角色分工，明確的責任邊界
- ✓ 標準化流程，減少重複溝通與等待
- ✓ 透明可視的進度，降低風險與遺漏
- ✓ 有紀錄、可追蹤、可稽核、可治理

帶來的正向改變

- 減少溝通成本，專注在創造價值的事
- 流程更順暢，交付更快
- 紀錄完整，知識可傳承
- 風險可控，安全可治理
- 團隊信任工作流，而不是互相信任

第二條價值流

從「Key Vault Ready」到「Secret Ready」

Key Vault Ready → Secret Ready



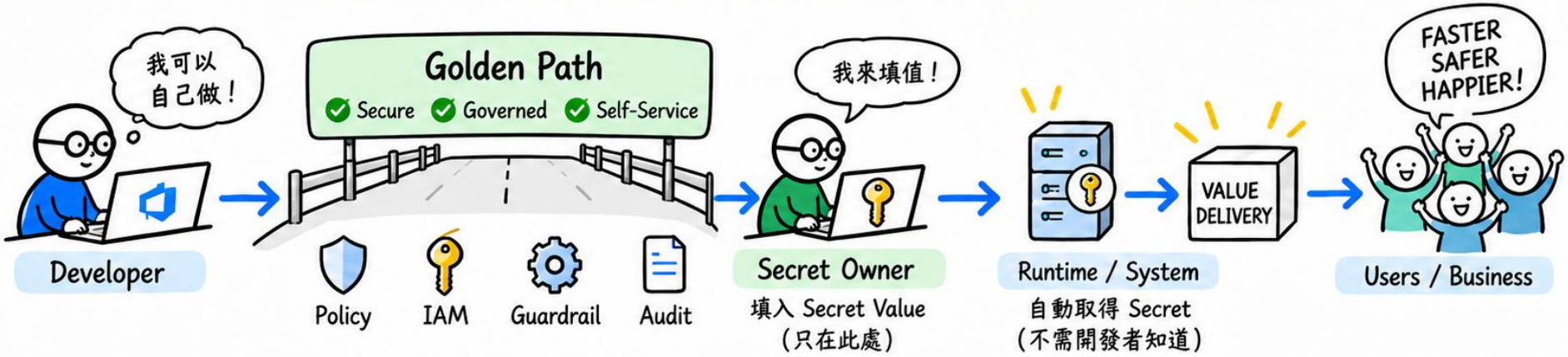
Metric	Before	Golden Path
Lead Time	~2 days	DEV/SIT：數分鐘；UAT/PROD：約 2 days
Human Handoff	Developer → DBA/SRE	仍然存在
Developer preparation	Work Item x 2	Contract / YAML
Secret Owner execution	解讀需求 → 找位置 → 填值	依 Contract 執行
DEV/SIT Self-Service	0%	**100%**
UAT/PROD	人工換手	刻意保留職責分離

清楚的職責分離與流程的改善，兼顧了機敏資訊治理與換手暢流設計。

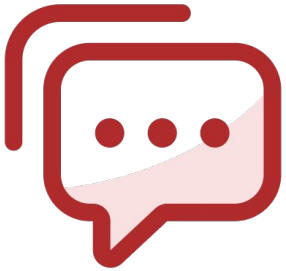
開發者不需要知道密碼，也能完成工作

只有被理解的流程，才值得被自動化。

不是讓更多人知道密碼，
而是讓每個人只接觸自己該接觸的資訊，
最後仍然能把價值安全地送出去。



Do not edit
How to change the design



Audience Q&A

Presenting with animations, GIFs or speaker notes? Enable our Chrome extension

slido